

MVC Request Lifecycle & Core Internals

Q1. Describe the complete lifecycle of a Spring MVC request from the moment it enters the server until a response is returned.

A: The request first hits the **DispatcherServlet**, which consults the **HandlerMapping** to find the correct Controller method. The request then passes through the **HandlerInterceptors**. After the Controller executes, the logical view name is returned, which the **ViewResolver** translates into a physical View. Finally, the View is rendered, and the response is sent back via the DispatcherServlet.

Q2. What responsibilities does the DispatcherServlet have in this lifecycle?

A: The **DispatcherServlet** acts as the front controller, receiving every request and managing the entire execution flow. Its core responsibilities include finding the right handler (Controller), coordinating the execution of interceptors, managing the flow of the Model, and ultimately delegating the rendering process to the ViewResolver.

Q3. How do controllers, handler mappings, interceptors, and view resolvers all integrate during request processing?

A: The **HandlerMapping** translates the request URL into a specific **Controller** method. **Interceptors** execute code before and after the Controller method runs. The Controller returns a logical view name, which the **ViewResolver** uses to locate the final rendering template. The **DispatcherServlet** orchestrates all these components.

Q4. What is the role of the WebApplicationContext, and how does it differ from the root ApplicationContext?

A: The **WebApplicationContext** is the Spring container specific to the MVC layer; it holds all web-related beans (Controllers, ViewResolvers). The **root ApplicationContext** is the parent container, typically holding business logic and persistence beans (Services, Repositories). The WebApplicationContext can access beans from the root context, but not vice versa, creating a clean separation of concerns.

MVC Bootstrapping, Java Config & Servlet Infrastructure

Q5. How do you configure a Spring MVC application in a traditional web environment?

A: In a traditional servlet environment, you configure Spring MVC using the **web.xml** file. You must define the **DispatcherServlet** entry and specify its mapping (e.g., mapping all requests

starting with `/`). You also define the **ContextLoaderListener** to load the root **ApplicationContext**.

Q6. What is the role of web.xml or Java-based configuration in setting up Spring MVC?

A: The role is to define the **entry points** for the application. The **web.xml** file (or its modern Java configuration equivalent) defines which servlet (the **DispatcherServlet**) handles which requests. It also specifies the location of the Spring configuration files or Java configuration classes needed to bootstrap the container.

Q7. How do you bootstrap a Spring MVC application without using a web.xml file?

A: You bootstrap the application using Java-based configuration by creating a class that implements the **WebApplicationInitializer** interface. This interface allows you to programmatically define and register the **DispatcherServlet** and its context configuration (which points to your `@Configuration` classes), replacing the need for the **web.xml** file.

Q8. What roles do DispatcherServlet, ContextLoaderListener, and servlet listeners play during application initialization?

A: The **ContextLoaderListener** initializes the root **ApplicationContext** (business/persistence layer) when the application starts. The **DispatcherServlet** is responsible for initializing the **WebApplicationContext** (web layer). Other servlet listeners execute custom code during specific stages of the servlet container's lifecycle (e.g., when the session starts).

View Resolution, Content Negotiation & Advanced Rendering

Q9. What are the advantages of using a ContentNegotiatingViewResolver in a Spring MVC application?

A: The primary advantage is flexibility. It allows a controller to return data (like a user object) without caring about the output format. It uses the requested content type (from the **Accept** header or a URL extension like `.json`) to dynamically choose the correct **ViewResolver** to handle the data serialization or view rendering.

Q10. How do multiple view resolvers work when registered in a chain, and how does Spring decide which one to use?

A: Multiple **ViewResolvers** work in a chain and are ordered using a priority number. Spring iterates through the chain; if a resolver successfully finds the view for the logical name, the

chain stops. If the resolver cannot find the view, it returns null, and Spring proceeds to the next resolver in the chain.

Interceptors & Advanced Handler Pre/Post Processing

Q11. How do you configure an interceptor so that it applies to all controllers globally?

A: You configure an interceptor globally by creating a configuration class that implements the **WebMvcConfigurer** interface. You then override the **addInterceptors()** method and use **registry.addInterceptor(new MyInterceptor())** without specifying a path. This automatically applies the interceptor to all incoming requests handled by the DispatcherServlet.

Q12. What are the phases of request processing within a HandlerInterceptor, and how do they impact the flow?

A: The phases are: **preHandle()** (runs *before* the controller; can stop the flow by returning false). **postHandle()** (runs *after* the controller but *before* view rendering; can modify the Model). **afterCompletion()** (runs *after* the entire process, including view rendering; used for cleanup).

Exception Handling (Advanced)

Q13. How does Spring MVC classify and route different types of exceptions?

A: Spring MVC classifies exceptions based on the type thrown. It routes exceptions using a hierarchy: first, to a local **@ExceptionHandler** method in the controller. If not found, it routes to a global handler defined by **@ControllerAdvice**. Finally, if still uncaught, it routes to the default error page handler.

Q14. What advanced options exist for exception handling beyond @ExceptionHandler? (e.g., @ControllerAdvice, custom resolvers)

A: Beyond **@ExceptionHandler** (local), you use **@ControllerAdvice** to centralize exception handling globally across multiple controllers. Another option is defining a custom **HandlerExceptionResolver** bean, which gives you programmatic control over handling the exception and generating the final response.

Security Integration with MVC

Q15. What security mechanisms can be used to protect a Spring MVC application?

A: You protect an application using the **Spring Security** framework. Mechanisms include **Authentication** (verifying identity), **Authorization** (checking roles/permissions), built-in defenses against **CSRF** (Cross-Site Request Forgery) and **SQL Injection**, and **TLS/HTTPS** encryption for all network traffic.

Q16. How does Spring Security integrate with MVC controllers and view rendering?

A: Spring Security integrates via a **Filter Chain** that runs before the `DispatcherServlet`. It establishes a `SecurityContext` for the request. It integrates with controllers via method-level annotations (`@PreAuthorize`) and integrates with view rendering by providing security tags to show or hide parts of the page based on the user's permissions.

Q17. What challenges commonly appear when securing MVC applications, and how do you address them?

A: A common challenge is securing data passed in the session (`@SessionAttributes`) and ensuring proper logout invalidates the session. You address **CSRF** by including the necessary token in all forms. You address **access control** by ensuring every URL is secured (defaulting to denial) and every management endpoint is protected by an explicit role check.

Q18. What are the configuration steps required to enable method-level security in a Spring MVC application?

A: The primary step is adding the `@EnableMethodSecurity` annotation to a configuration class. Once enabled, you can use annotations like `@PreAuthorize` (check before execution) and `@PostAuthorize` (check after execution) directly on your controller or service methods.

Data Binding, Custom Editors & Validation

Q19. How do you customize data binding for nested/complex objects in Spring MVC?

A: You customize data binding by implementing a custom **PropertyEditor** or **Converter** interface. This allows you to define logic for translating complex string input (like a formatted date or time) from the request parameters into a specific Java object type (like `java.time.LocalDate`) that is needed by the controller.

Q20. What issues commonly arise during data binding, and how do you mitigate them?

A: Common issues include **Type Mismatch** errors (e.g., a user enters a non-numeric string into an int field) and **Missing Parameters**. You mitigate them by using the **@InitBinder** annotation to register custom PropertyEditors for type conversion and by checking the **BindingResult** object for validation errors in the controller method.

Static Resources, Performance & Resource Handling

Q21. What performance considerations should you keep in mind when serving static resources (CSS/JS/images)?

A: You should prioritize **caching** static resources effectively by setting appropriate Cache-Control headers (long expiry times). You should also configure Spring MVC to serve these resources directly from the classpath (/static/), and potentially use a dedicated Content Delivery Network (CDN) for production traffic.

Q22. How does Spring's resource handling differ from standard servlet container behavior?

A: Standard servlet containers serve resources from the deployed directory structure. Spring's resource handling allows you to serve static files directly from the **classpath** (e.g., from src/main/resources/static). This is beneficial for packaging as a single JAR and simplifies versioning and caching by adding hashes to the resource URLs.

URL Design, Path Variables & Routing Complexity

Q23. What should you consider when designing URLs that use @PathVariable?

A: You should consider making the URLs **resource-centric** (using nouns) and clear. Path variables should be used to specify the exact resource ID (e.g., /users/{id}). The variable name should clearly reflect the data type expected (e.g., /{userId}), and you should ensure the variable type in the controller method signature matches the expected input.

Q24. How does @PathVariable interact with overlapping or ambiguous request mappings?

A: When mappings overlap (e.g., /users/new and /users/{id}), Spring uses a sophisticated ranking system to resolve the ambiguity. Spring prioritizes the **most specific** match, meaning

the static path (/users/new) is always chosen over the dynamic path variable (/users/{id}). If ambiguity persists, Spring throws an exception at startup.

Internationalization (i18n)

Q25. How does Spring MVC use a LocaleResolver to support multiple languages?

A: The **LocaleResolver** is responsible for determining the user's current locale (language and region). It typically inspects the HTTP request (e.g., checking the Accept-Language header or a specific cookie) to resolve the user's preferred language, which is then used by the application to display corresponding messages and currency formats.

Q26. How would you allow users to switch languages dynamically from the frontend?

A: You allow dynamic language switching by implementing an interceptor (a **LocaleChangeInterceptor**). This interceptor detects a change parameter (e.g., ?lang=fr) in the URL and instructs the LocaleResolver (like a CookieLocaleResolver) to update and save the user's language preference for future requests.

Sessions, Cookies & Advanced MVC Annotations

Q27. How do annotations like @SessionAttributes and @CookieValue function in Spring MVC?

A: **@SessionAttributes** marks specific Model attributes that should be temporarily stored in the HTTP session between requests, often used during a multi-step form process. **@CookieValue** extracts the value of a specific cookie sent by the client and maps it directly to a controller method parameter.

Q28. What security concerns arise when using @SessionAttributes or cookie-based data access?

A: Security concerns include **data exposure** (if sensitive data is stored in the session or cookie) and **session fixation/hijacking**. Mitigation involves using **HTTPS** to encrypt cookies, setting the **Secure** and **HttpOnly** flags on cookies, and ensuring no unauthorized data is placed in the session.

Testing (Advanced Mocking + Integration)

Q29. How do you mock MVC dependencies when unit testing controllers?

A: You unit test controllers by creating a test class annotated with **@WebMvcTest**. You mock the controller's dependencies (Services, Repositories) by injecting them as **@MockBean** fields. This allows the test to focus solely on the Controller's routing and input/output behavior without hitting the actual business logic.

Q30. What are best practices for writing integration tests for Spring MVC applications?

A: Best practices include using **@SpringBootTest** to load the full application context and using the **MockMvc** utility to simulate HTTP requests. You should use an in-memory database (like H2) and ensure the tests clean up any data created, promoting isolation and fast execution.

File Upload Handling

Q31. What configurations are required to enable file uploads in a Spring MVC application?

A: The required configuration is primarily ensuring the **commons-fileupload** library is on the classpath. In Spring Boot, this is often handled automatically. You must also configure the **MultipartResolver** bean to handle the encoding, although Spring Boot usually auto-configures the **StandardServletMultipartResolver**.

Q32. How do you handle multi-part file uploads within a controller?

A: You handle multi-part files by defining a controller method that accepts a parameter of type **MultipartFile**. This object contains the file content, original filename, and size. You access the file data by calling methods like `file.getBytes()` or `file.getInputStream()`.

Q33. What common file-upload issues arise in production, and how do you resolve them?

A: Common issues include **OutOfMemory Errors** (when attempting to buffer large files into the heap) and **Disk Space Exhaustion**. You resolve these by configuring the maximum file size limit and by ensuring large files are handled via **streaming** (`InputStream`) directly to persistent storage (like S3) instead of buffering in memory.

Integrations with Other Technologies

Q34. How can a Spring MVC application be integrated with persistence technologies like JPA/Hibernate?

A: Integration is achieved by defining **@Service** components that encapsulate the business logic and **@Repository** components that use **Spring Data JPA** interfaces. The Service layer calls the Repository, which handles persistence operations via Hibernate or JPA, and the Controller injects and calls the Service.

Q35. How can Spring MVC work together with WebSockets to support real-time features?

A: Spring MVC uses its own dedicated infrastructure (the DispatcherServlet) for traditional HTTP requests. **WebSockets** use a different protocol and are typically implemented using Spring's **STOMP** (Simple Text Oriented Messaging Protocol) message broker support. The MVC controllers can trigger WebSocket messages, allowing HTTP requests to initiate real-time communication.